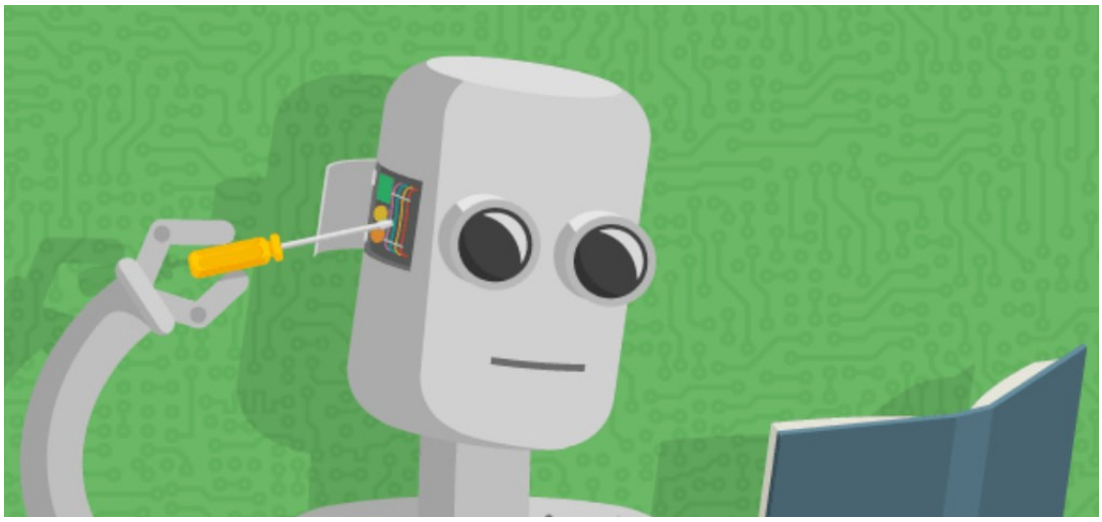
# Supervised Learning



**Mehrtash Harandi**[*] MEHRTASH.HARANDI@MONASH.EDU
*Department of Electrical and Computer Systems Engineering, Monash University*

[*] These notes are meant to supplement the lectures for ECE4076/5176 at Monash University given by the author. The author makes no guarantees that these notes are free of typos or other, more serious errors.

© M. Harandi.

## 1. Introduction

Supervised learning addresses the task of **predicting targets given input data**. While it is just one among several paradigms within machine learning, supervised learning accounts for the majority of successful applications of machine learning in industry. Partly, that is because many important tasks can be described crisply as estimating the probability of something unknown given a particular set of available data. For example:

- Predict cancer vs. not cancer, given a CT image.
- Predict whether an image is a cat or a dog.
- Predict the correct translation in French, given a sentence in English.

In supervised learning, the targets are often called **labels** and generally denoted by $y$. The input data, also called the features, are typically denoted by $\boldsymbol{x}$. Each (input, target) pair is called an example or instance. We denote any particular instance with a subscript, typically $i$ , for instance $(\boldsymbol{x}_i, y_i)$. A dataset $\mathcal{D}$ is a collection of $m$ instances $\{\boldsymbol{x}_i, y_i\}_{i=1}^{m}$[1]. Our goal is to design a model (or if you want a function) $f_\theta$ with parameters $\theta$ that maps any input $\boldsymbol{x}_i$ to a prediction $f_\theta(\boldsymbol{x}_i)$ to match the target $y_i$ (according to some measures).

**Example 1** *Suppose you want to predict whether or not a patient would have a heart attack. This observation, heart attack or no heart attack, would be our label $y$. The input data $\boldsymbol{x}$ might be vital signs such as heart rate, diastolic and systolic blood pressure, etc.*

**Example 2** *We have a set of data harvested from a database of home sales in the form of a table, where each row corresponds to a different house, and each column corresponds to a relevant attribute, such as the square footage of a house, the number of bedrooms, the number of bathrooms, and the walking time (minutes) to the center of town. If you live in a metropolitan city such as Melbourne, and not owing a gold mine, the (sq. footage, no. of bedrooms, no. of bathrooms, walking distance) feature vector for your home might look something like:* $[100, 2, 2, 20]$. *However, if you live in rural, it might look more like* $[3000, 8, 5, 45]$. *Feature vectors like this are essential for most classic machine learning algorithms.*

The term *supervision* comes into play because for choosing the parameters $\boldsymbol{w}$ of the model, we provide it with a dataset consisting of labeled examples $(\boldsymbol{x}_i, y_i)$, where each example $\boldsymbol{x}_i$ is matched with the correct label. Even with the simple description predict targets from inputs, supervised learning can take various forms and require a great many modeling decisions, depending on (among other considerations) the type, size, and the number of inputs and outputs. For example, we use different models to process sequences (like strings of text or time series data) and for processing fixed-length vector representations.

[1] read this as a set with $m$ pairs of the form $(\boldsymbol{x}_i, y_i)$.

**A high-level picture.** Informally, the learning process for supervised learning looks something like this: grab a big enough collection of training examples for which the inputs $\boldsymbol{x}_i$ and their associated labels $y_i$ are known. Feed the dataset into a supervised learning algorithm, a procedure that takes as input a dataset and outputs a function, the learned model. The learning algorithm has some parameters $\boldsymbol{w}$ and by seeing the set of input/target data, it changes its parameters to best describe the data for us (according to its objective). Finally, you can feed previously unseen inputs to the learned model, using its outputs as predictions of the corresponding label. The full process is drawn in Figure 1.

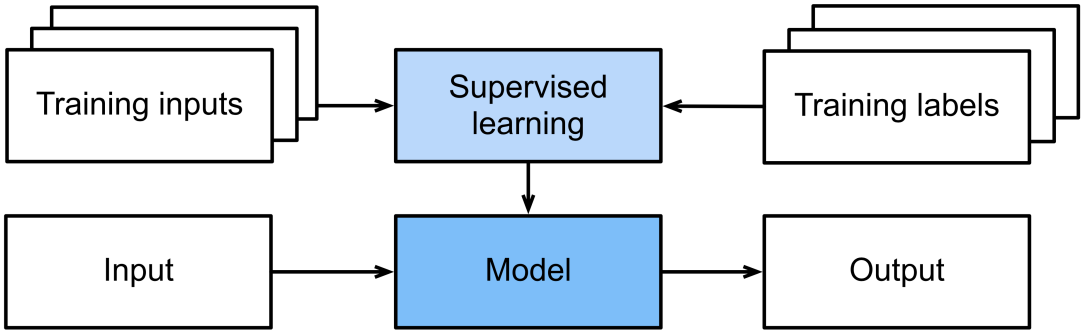



Figure 1: Supervised learning (figure credit **?**)

### 1.1. Regression

Perhaps the simplest supervised learning task to wrap your head around is regression. What makes a supervised learning problem a regression is actually the type of the outputs/labels. When our targets take on arbitrary values in some range, we call this a regression problem (*i.e.*, if you have $y \in \mathcal{S} \subset \mathbb{R}$). For example, suppose you want to design a model to predict the temperature of the next day in Melbourne. In this case, your model should predict values in the range $[-20, 55]$ (you may argue that this range is loose which is a fair statement). Our goal is to produce a model whose predictions closely approximate the actual target values. We denote the predicted target for an instance $x_i$ by $\hat{y}_i$. Do not worry if the notation is bogging you down, we will make things clear throughout the course.

Lots of practical problems are well-described regression problems. In computer vision, predicting the location of objects (called object detection), predicting the human pose, predicting the facial landmarks are all examples of a regression problem. A good rule of thumb is that any "How much?" or "How many?" problem should suggest regression.

Even if you have never worked with machine learning before, you have probably worked through a regression problem informally. Imagine, for example, that you had your lawn mowed and that your contractor spent $x_1 = 3$ hours to do the job. Then he sent you a bill of $y_1 = 350\$$. Now imagine that your friend hired the same contractor for $x_2 = 2$ hours and that she received a bill of $y_2 = 250\$$. If someone then asked you how much to expect on their upcoming invoice you might make some reasonable assumptions, such as more hours worked costs more dollars. You might also assume that there is some base charge and that

the contractor then charges per hour. If these assumptions held true, then given these two data points, you could already identify the contractor's pricing structure: 100$ per hour plus 50$ to show up at your house. If you followed that much then you already understand the high-level idea behind linear regression (and you just implicitly designed a linear model with a bias term).

### 1.2. Classification

While regression models are great for addressing "how many?" questions, lots of problems do not fit comfortably to this template. For example, a bank wants to add check scanning to its mobile app. This would involve the customer snapping a photo of a check with their smart phone's camera and the machine learning model would need to be able to understand hand-written text. This kind of system is referred to as optical character recognition (OCR), and the kind of problem it addresses is called classification. In short, if you can easily pose your problem as "Is this a $\cdot$?", then it is likely, classification. Classification is treated with a different set of algorithms than those used for regression (although many techniques will carry over).

In classification, we want our model to look at a feature vector (*e.g.*, the pixel values in an image) and then predict which category (formally called classes), among some (discrete) set of options, an example belongs to. For hand-written digits, we might have 10 classes, corresponding to the digits 0 through 9. The simplest form of classification is when there are only two classes, a problem which we call binary classification. For example, our dataset $\mathcal{D}$ could consist of images of animals and our labels $\mathcal{Y}$ might be the classes $\{\text{cat}, \text{dog}\}$. While in regression, we sought a regressor to output a real value $\hat{y}$, in classification, we seek a classifier, whose output $\hat{y}$ is the predicted class assignment.

For reasons that we will get into later, it can be hard to optimize a model that can only output a hard categorical assignment (*e.g.*, either cat or dog). In these cases, it is usually much easier to instead express our model in the language of probabilities. Given an example $\boldsymbol{x}$, our model assigns a probability $\hat{y}_k$ to each label $k$. Because these are probabilities, they need to be positive numbers and add up to one and thus we only need $K-1$ numbers to assign probabilities of $K$ categories. This is easy to see for binary classification. If there is a 0.6 (60%) probability that an unfair coin comes up heads, then there is a 0.4 (40%) probability that it comes up tails. Returning to our animal classification example, a classifier might see an image and output the probability that the image is a cat $\mathbb{P}(\hat{y} = \text{cat}|\boldsymbol{x}) = 0.9$. We can interpret this number by saying that the classifier is 90% sure that the image depicts a cat. The magnitude of the probability for the predicted class conveys a notion of uncertainty.

When we have more than two possible classes, we call the problem multiclass classification. Common examples include hand-written character recognition $[0, 1, 2, 3, \cdots, 9, a, b, c, \cdots, z]$.

## 2. Linear Regression

Let $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \cdots, (x_m, y_m)\}$ be $m$ points with $x_i, y_i \in \mathbb{R}$. To predict $y$ for a new sample $x_q$, we make use of a linear model with parameters $w$ and $b \in \mathbb{R}$ in the form

$$\hat{y} = wx + b \,. \tag{2.1}$$

Our goal is to obtain the value of parameters $w$ and $b$ that best describe our input data $\mathcal{D}$, hoping that such a linear model that describes our input data well, will predict reasonably well for $x_q$. One approach for solving this problem is to define an error function and minimize it to obtain the values of $w$ and $b$. In linear regression, we define the loss (*i.e.*, error) of the model as

$$\begin{aligned}\mathcal{L}_{\text{SE}}(w,b) &\triangleq \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 \\ &= \frac{1}{m}\sum_{i=1}^{m}(wx_i + b - y_i)^2 \,. \end{aligned} \tag{2.2}$$

Let's study Equation (2.2) to better understand the underlying idea. From Equation (2.2), we notice that the error of prediction for data point $x_i$ is $(wx_i+b-y_i)^2$. That is, our model, once we have the optimal value of $w$ and $b$, will predict $wx_i + b$ as its output for $x_i$. We want this output to be close, as much as possible, to $y_i$ which is our desired output. We measure the closeness of $\hat{y}_i = wx_i+b$ and $y_i$ using squared distances. The choice of squared distances is important in linear regression as it provides us with a very simple solution that can be generalized to multivariate cases easily. That said, in some cases you may want to opt for other closeness measures (*e.g.*, $|\cdot|$ for 1D data and $\|\cdot\|$ for multivariate case[2]), but this is beyond the scope of our course.
The next thing to pay attention to is the fact that instead of looking at one single data point, we would like to find $w$ and $b$ that minimize a total prediction error, hence the summation. To obtain the optimal values of the parameters $w$ and $b$, we need to solve[3]

$$\frac{\partial \mathcal{L}_{\text{SE}}(w,b)}{\partial w} = 0. \tag{2.3}$$

$$\frac{\partial \mathcal{L}_{\text{SE}}(w,b)}{\partial b} = 0. \tag{2.4}$$

Define

$$\hat{\sigma}_{xx}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \bar{x})^2,$$

$$\hat{\sigma}_{xy}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \bar{x})(y_i - \bar{y}),$$

[2]Recall that the (dis)similarity between two points on the real line can be measured as

$$d(p,q) = |p-q|,\ \forall p,q \in \mathbb{R}\,.$$

If $\boldsymbol{p},\boldsymbol{q} \in \mathbb{R}^n$, then we define their Euclidean distance as

$$d(\boldsymbol{p},\boldsymbol{q}) = \|\boldsymbol{p}-\boldsymbol{q}\| = \sqrt{\sum_{i=1}^{n}\left|\boldsymbol{p}[i]-\boldsymbol{q}[i]\right|^2}\,,$$

where the notation $\boldsymbol{a}[i]$ denotes element at location $i$ in the vector $\boldsymbol{a}$.

[3]The notation $\partial$ means partial derivative.

with

$$\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i,$$
$$\bar{y} = \frac{1}{m}\sum_{i=1}^{m} y_i.$$

Then the optimal values for $w$ and $b$, denoted by $w^*$ and $b^*$ by solving Equation (2.3) and Equation (2.4) are (see Appendix A.1 for the details).

$$w^* = \frac{\hat{\sigma}_{xy}^2}{\hat{\sigma}_{xx}^2}, \tag{2.5}$$
$$b^* = \bar{y} - \bar{x}w^*. \tag{2.6}$$

An interesting observation here is this. If we assume that $x$ and $y$ are random variables with certain properties (again here we will not delve into statistics for the sake of simplicity), $\bar{x}$, $\bar{y}$, $\hat{\sigma}_{xx}^2$, and $\hat{\sigma}_{xy}^2$ are the sample mean of $x$, $y$, the variance of $x$ and the covariance of $x$ and $y$, respectively. Suppose $x$ and $y$ are not related. In this case, $\sigma_{xy} \simeq 0$, and hence $w^* = 0$ and $b^* = \bar{y}$. In other words, when there is no connection between $x$ and $y$ (this is reflected by $\sigma_{xy} \simeq 0$), the best estimation for $y$ is its mean which makes perfect sense. Algorithm 1 details out the univariate linear regression method.

**Algorithm 1:** Univariate Linear Regression

**input :** $m$ data pairs, $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \cdots, (x_m, y_m)\}$ with $x_i \in \mathbb{R}$ and $y_i \in \mathbb{R}$. Here, $y_i$ is the desired output for $x_i$.

**output:** $w^*$ and $b^*$, the parameters of the linear model $\hat{y} = w^*x + b^*$

$\bar{x} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$;

$\bar{y} \leftarrow \frac{1}{m}\sum_{i=1}^{m} y_i$;

$\hat{\sigma}_{xx}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \bar{x})^2$;

$\hat{\sigma}_{xy}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \bar{x})(y_i - \bar{y})$;

$w^* \leftarrow \hat{\sigma}_{xy}^2/\hat{\sigma}_{xx}^2$;

$b^* \leftarrow \bar{y} - \bar{x}w^*$;

## 2.1. Multivariate Linear Regression

We now take a further step and generalize the previous result. First, we assume that we are interested in predicting a scalar from multivariate inputs. To be precise, our assumption is now that the data is in the form $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_m, y_m)\}$, where $\boldsymbol{x}_i \in \mathbb{R}^n$ and

$y_i \in \mathbb{R}$. Our linear regression model takes the following form[4]

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} \; . \tag{2.7}$$

For reasons that become clear soon, we would not consider a bias term here for now. We define the loss (*i.e.*, error) of our model as:

$$\begin{aligned} \mathcal{L}_{\text{SE}}(\boldsymbol{w}) &\triangleq \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2 \\ &= \frac{1}{m} \sum_{i=1}^{m} (\boldsymbol{w}^\top \boldsymbol{x}_i - y_i)^2 \; . \end{aligned} \tag{2.8}$$

To obtain $\boldsymbol{w}^*$, the minimizer of $\mathcal{L}_{\text{SE}}(\boldsymbol{w})$, we need to solve

$$\nabla_{\boldsymbol{w}} \mathcal{L}_{\text{SE}} = 0, \tag{2.9}$$

where $\nabla_{\boldsymbol{w}} \mathcal{L}_{\text{SE}}(\boldsymbol{w}) \in \mathbb{R}^n$ is the $n$ dimensional gradient of $\mathcal{L}_{\text{SE}}(\boldsymbol{w})$ with respect to $\boldsymbol{w}$ and is defined as;

$$\nabla_{\boldsymbol{w}} \mathcal{L}_{\text{SE}} = \begin{pmatrix} \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{w})}{\partial w_1} \\ \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{w})}{\partial w_2} \\ \vdots \\ \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{w})}{\partial w_n} \end{pmatrix} . \tag{2.10}$$

The solution to Equation (2.9) is given by (see Section A.2 for details);

$$\boldsymbol{w}^* = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}, \tag{2.11}$$

where

$$\boldsymbol{X} = \begin{pmatrix} x_1[1] & x_1[2] & \cdots & x_1[n] \\ x_2[1] & x_2[2] & \cdots & x_2[n] \\ \vdots & \vdots & & \vdots \\ x_m[1] & x_m[2] & \cdots & x_m[n] \end{pmatrix} , \tag{2.12}$$

and

$$\boldsymbol{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} . \tag{2.13}$$

[4] We denote the transpose of a vector or a matrix by $\top$. Hence, $\boldsymbol{a}^\top \boldsymbol{b}$ denotes the inner or dot product between two vectors $\boldsymbol{a}$ and $\boldsymbol{b}$.

Here, $x_i[j]$ denotes the j-th elements of $x_i$ (recall that $\boldsymbol{x}_i$ is a n-dimensional vector). To be specific, to obtain the parameters of our model, we first stack all the $m$ inputs into rows of the $m \times n$ matrix $\boldsymbol{X}$. We then form an $m$-dimensional vector by stacking our desired responses into $\boldsymbol{y}$. The matrix $\boldsymbol{X}^\top \boldsymbol{X}$ is a symmetric matrix of size $n \times n$ and hence the dimensionality of $\left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top$ is $n \times m$[5]. Since the dimensionality of $\boldsymbol{y}$ is $m \times 1$, the dimensionality of the form $\left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$ is $n \times 1$, exactly the dimensionality of our parameters. Once we have $\boldsymbol{w}^*$, to predict $y_q$ for a new sample $\boldsymbol{x}_q$, we simply compute $y_q = \boldsymbol{w}^{*\top} \boldsymbol{x}_q$. Algorithm 2 details out the linear regression method for multivariate input data.

**Algorithm 2:** Multivariate Linear Regression

**input :** $m$ data pairs, $\mathcal{D} = \left\{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_m, y_m)\right\}$ with $\boldsymbol{x}_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$. Here, $y_i$ is the desired output for $\boldsymbol{x}_i$.

**output:** $\boldsymbol{w}^*$, the parameters of the linear model $\hat{y} = \boldsymbol{w}^{*\top} \boldsymbol{x}$

Form $\boldsymbol{X}$ according to Equation (2.12);

Form $\boldsymbol{y}$ according to Equation (2.13);

$\boldsymbol{w}^* \leftarrow \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$;

**Remark 1 (Moore-Penrose inverse)** *The matrix* $\boldsymbol{X}^+ \triangleq \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top$ *is called the Moore-Penrose inverse or pseudoinverse of* $\boldsymbol{X}$ *as* $\boldsymbol{X}^+ \boldsymbol{X} = \mathbf{I}$. *Note that, the inverse is only defined for square matrices. Further, not all square matrices come with an inverse. The pseudoinverse enables us to invert a non-square matrix and exists under mild conditions*[6] *and is unique. Now a simple way to remember the linear regression is to say: my data follows a linear model as* $\boldsymbol{y}_{m\times 1} = \boldsymbol{X}_{m\times n} \boldsymbol{w}_{n\times 1}$ *and hence*

$$\begin{aligned} \boldsymbol{y}_{m\times 1} &= \boldsymbol{X}_{m\times n} \boldsymbol{w}_{n\times 1} \\ \Rightarrow \boldsymbol{X}^+ \boldsymbol{y} &= \underbrace{\boldsymbol{X}^+ \boldsymbol{X}}_{\mathbf{I}} \boldsymbol{w} \Rightarrow \boxed{\boldsymbol{w} = \boldsymbol{X}^+ \boldsymbol{y}} \,. \end{aligned}$$

**Overdetermined and Underdetermined.** Recall that the dimensionality of $\boldsymbol{X}$ is $m \times n$ (or if we consider the bias, $m \times (n+1)$). Consider the linear form

$$\boldsymbol{X}_{m\times n} \boldsymbol{w}_{n\times 1} = \boldsymbol{y}_{m\times 1} \,.$$

You can interpret this as a set of $m$ linear equations for $n$ unknown variables $w_1, w_2, \cdots, w_n$. If $m > n$ (*i.e.*, the number of samples exceeds the dimensionality of samples), we have more equations than unknowns. We call this an **overdetermined system**. In general and in an overdetermined system, the equations will be inconsistent and instead of demanding that $\boldsymbol{X}\boldsymbol{w} = \boldsymbol{y}$, we look for a $\boldsymbol{w}$ than makes the difference between $\boldsymbol{X}\boldsymbol{w}$ and $\boldsymbol{y}$ as small as possible, as measured by the $\ell_2$ norm. That is, we pick $\boldsymbol{w}$ to minimize

$$\left\| \boldsymbol{X}\boldsymbol{w} - \boldsymbol{y} \right\|^2 .$$

[5] Recall from linear algebra that the multiplication of two matrices, $\boldsymbol{A}_{r\times s}$ of size $r \times s$ and $\boldsymbol{B}_{s\times t}$ of size $s \times t$ is a matrix of size $r \times t$.

[6] The requirement is the existence of the inverse of the square and symmetric matrix $\left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1}$, which is a simpler condition.

In other words, we solve the system as best as we can, according to the $\ell_2$ norm.
If $m \leq n$, then $\boldsymbol{X}^\top \boldsymbol{X}$ is singular and does not have an inverse. Mathematically, it is similar to seek solving a system of $m$ linear equation where the number of variables exceeds the number of equations. We call such a system an **underdetermined system**. Such a system has infinite number of solutions, so does our MV regression problem. One of the solutions in such a case can be written as

$$\boldsymbol{w}^* = \boldsymbol{X}^\top \left(\boldsymbol{X}\boldsymbol{X}^\top\right)^{-1} \boldsymbol{y}. \tag{2.14}$$

### 2.2. How to Add Bias to our Model?

Recall that for univariate linear regression, our model has a bias term $b$, *i.e.*,

$$\hat{y} = wx + b.$$

In contrast, in the multivariate case, we use a model in the form

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x}.$$

One may wonder why a bias term is not considered in the multivariate case. The answer to such a valid concern is that the form $\boldsymbol{w}^\top \boldsymbol{x}$ can readily incorporates a bias term. In doing so, we define an augmented version of our samples as,

$$\boldsymbol{x}_{\text{aug}} = \begin{pmatrix} 1 \\ x[1] \\ x[2] \\ \vdots \\ x[n] \end{pmatrix}. \tag{2.15}$$

In essence, we add a deterministic feature with value 1 to each sample (just $\boldsymbol{x}$ and not $y$). This increases the dimensionality of the input from $n$ to $n+1$ and hence the linear regression model will have $n+1$ parameters now. With this and indexing the elements of a vector from zero (the same way Python and C consider indexes), let's expand the linear form

$$\begin{aligned} \boldsymbol{w}^\top \boldsymbol{x}_{\text{aug}} = \sum_{i=0}^{n} w_i x_{\text{aug}}[i] &= w_0 x_{\text{aug}}[0] + \sum_{i=1}^{n} w_i x_{\text{aug}}[i] \\ &= w_0 \times 1 + \sum_{i=1}^{n} w_i x_{\text{aug}}[i]. = \underbrace{w_0}_{b} + \sum_{i=1}^{n} w_i x_{\text{aug}}[i]. \end{aligned} \tag{2.16}$$

Recall that $x_{\text{aug}}[0] = 1$ (see Figure 2 for an illustration). This implies that by simply augmenting data with a deterministic feature of one, we can incorporate the bias into our model (the bias term is the zero-th element of $\boldsymbol{w}$. This reparameterization trick is widely used in linear models and enables us to simplify our derivations while not losing any modeling capacity.

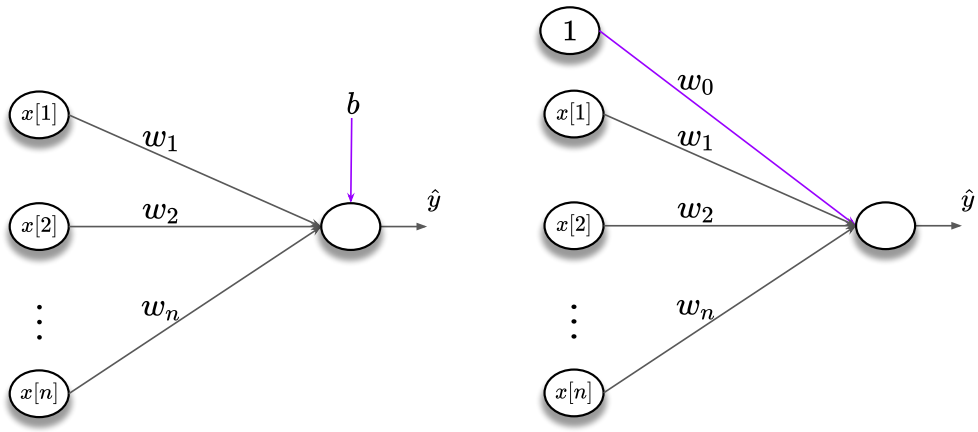



Figure 2: In linear models, we can omit the bias term from our derivations (for the sake of simplicity) and instead augment the data to implicitly consider it. In the left panel, we visualize the machinery of a univariate linear regression model. Therein, $z = \boldsymbol{w}^\top \boldsymbol{x} + b$. If we augment the data to have $\boldsymbol{x}_{\text{aug}} = (1, \boldsymbol{x})^\top$, then $w_0$ captures and represents the bias term.

### 2.3. Multivariate Outputs

In many applications, you may want to predict several outputs given your data. We give two examples below. The problem of object detection is defined as determining the coordinates of bounding boxes around objects in an image or video. You may also want to predict the class of each object along the bounding boxes (see Figure 3). As such, in object detection, your output might be a 4D vector representing the top-left corner of the bounding box, its width and height along with a label and the confidence of your prediction (per object in the image). In the problem of pose estimation, you are interested in identifying the location of various joints in an image (see Figure 4). In this case, your model should generate a vector with the coordinates of the joints (*e.g.*, arms, legs) per a person. Such systems can be used to analyze athletes performances among many other applications

We now use a very similar mechanism as to that of Section 2.1 to address multivariate outputs. More specifically, suppose our target values $\boldsymbol{y}$ are now $p$-dimensional vectors. The data in this case is in the form $\mathcal{D} = \left\{(\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \cdots, (\boldsymbol{x}_m, \boldsymbol{y}_m)\right\}$ with $\boldsymbol{x}_i \in \mathbb{R}^n$ and $\boldsymbol{y}_i \in \mathbb{R}^p$. We can define our linear regression model as:

$$\hat{y} = \boldsymbol{W}^\top \boldsymbol{x} \,, \tag{2.17}$$

where $\boldsymbol{W}$ is now an $n \times p$ matrix. Following our discussions in Section 2.1, we define the loss of our model as:

$$\mathcal{L}_{\text{SE}}(\boldsymbol{W}) \triangleq \frac{1}{m} \sum_{i=1}^{m} \|\hat{\boldsymbol{y}}_i - \boldsymbol{y}_i\|^2 = \frac{1}{m} \sum_{i=1}^{m} \left\|\boldsymbol{W}^\top \boldsymbol{x}_i - \boldsymbol{y}_i\right\|^2 . \tag{2.18}$$

To obtain $\boldsymbol{W}^*$, the minimizer of $\mathcal{L}_{\text{SE}}(\boldsymbol{W})$, we need to solve

$$\nabla_{\boldsymbol{W}} \mathcal{L}_{\text{SE}} = 0, \tag{2.19}$$

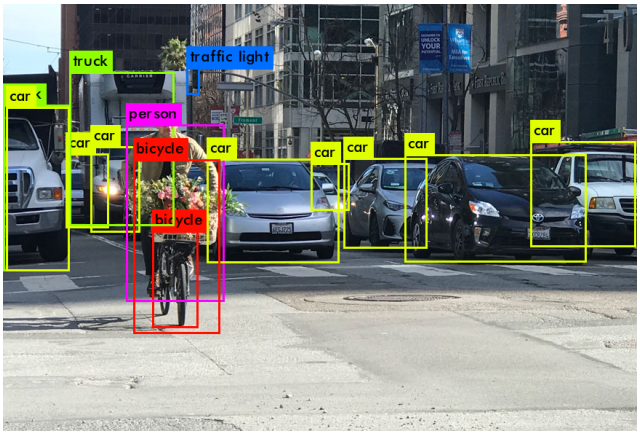



Figure 3: Object detection is an example of a problem with multiple outputs.

Figure 4: Pose estimation is another example where the output is multivariate.

where $\nabla_{\boldsymbol{W}} \mathcal{L}_{\text{SE}} \in \mathbb{R}^{n \times p}$ is the gradient of $\mathcal{L}_{\text{SE}}(\boldsymbol{W})$ with respect to $\boldsymbol{W}$ and is defined as;

$$\nabla_{\boldsymbol{w}} \mathcal{L}_{\text{SE}} = \begin{pmatrix} \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{W})}{\partial W_{1,1}} & \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{W})}{\partial W_{1,2}} & \cdots & \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{W})}{\partial W_{1,p}} \\ \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{W})}{\partial W_{2,1}} & \ddots & & \vdots \\ \vdots & & \ddots & \\ \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{W})}{\partial W_{n,1}} & \cdots & & \frac{\partial \mathcal{L}_{\text{SE}}(\boldsymbol{W})}{\partial W_{n,p}} \end{pmatrix}. \tag{2.20}$$

A closed form solution to Equation (2.19) is again given by:

$$\boldsymbol{W}^* = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{Y}, \tag{2.21}$$

where

$$\boldsymbol{X} = \begin{pmatrix} x_1[1] & x_1[2] & \cdots & x_1[n] \\ x_2[1] & x_2[2] & \cdots & x_2[n] \\ \vdots & \vdots & & \vdots \\ x_m[1] & x_m[2] & \cdots & x_m[n] \end{pmatrix}, \tag{2.22}$$

and

$$\boldsymbol{Y} = \begin{pmatrix} y_1[1] & y_1[2] & \cdots & y_1[p] \\ y_2[1] & y_2[2] & \cdots & y_2[p] \\ \vdots & \vdots & & \vdots \\ y_m[1] & y_m[2] & \cdots & y_m[p] \end{pmatrix}, \tag{2.23}$$

Here, $x_i[j]$ denotes the j-th elements of $x_i$ (recall that $\boldsymbol{x}_i$ is a n-dimensional vector). To be specific, to obtain the parameters of our model, we first stack all the $m$ inputs into rows of the $m \times n$ matrix $\boldsymbol{X}$. We then form an $m \times p$-dimensional matrix by stacking our desired responses into $\boldsymbol{Y}$. Then we obtain the parameters of our model according to Equation (2.21). Once we have $\boldsymbol{W}^*$, to predict $y_q$ for a new sample $\boldsymbol{x}_q$, we simply compute $\boldsymbol{y}_q = \boldsymbol{W}^{*\top}\boldsymbol{x}_q$. Algorithm 3 details out the method. In Figure 5, we illustrate the structure of a multivariate linear regression with multivariate outputs.

**Algorithm 3:** Multivariate Linear Regression (multivariate input/output)

**input :** $m$ data pairs, $\mathcal{D} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \cdots, (\boldsymbol{x}_m, \boldsymbol{y}_m)\}$ with $\boldsymbol{x}_i \in \mathbb{R}^n$ and $\boldsymbol{y}_i \in \mathbb{R}^p$. Here, $y_i$ is the desired output for $\boldsymbol{x}_i$.

**output:** $\boldsymbol{W}^*$, the parameters of the linear model $\hat{\boldsymbol{y}} = \boldsymbol{W}^{*\top}\boldsymbol{x}$

Form $\boldsymbol{X}$ according to Equation (2.22);

Form $\boldsymbol{Y}$ according to Equation (2.23);

$\boldsymbol{W}^* \leftarrow \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{Y}$;

## 3. Logistic Regression

Consider a binary classification problem. We have a set of training examples

$$\mathcal{D} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_m, y_m)\},$$

where $\boldsymbol{x}_i \in \mathbb{R}^n$ and $y_i \in \{0, 1\}$. Following our discussion in Section 2, we would like to design a model with parameters $\boldsymbol{w}$ that can predict the values of $y$ from $\boldsymbol{x}$. You may now tell yourself, the linear regression can be used to predict a scalar as the output so it fits to the setup of a binary classification problem. While nothing will stop you from doing this, the linear regression is not a very suitable choice here. For one, convince yourself that a linear regression model cannot map all samples exactly to $\{0, 1\}$[7]. To be able to address the classification problem, we need a few more tools at our disposal.

[7] Think about $\boldsymbol{x} \in \mathbb{R}^2$. Assume you have two samples, $\boldsymbol{x}_1, \boldsymbol{x}_2$ in class 0 and two examples $\boldsymbol{x}_3, \boldsymbol{x}_4$ in class 1. Can you find $\boldsymbol{w}$ such that it guarantees $\boldsymbol{w}^\top \boldsymbol{x}_1 = \boldsymbol{w}^\top \boldsymbol{x}_2 = 0$ and $\boldsymbol{w}^\top \boldsymbol{x}_3 = \boldsymbol{w}^\top \boldsymbol{x}_4 = 1$ ?

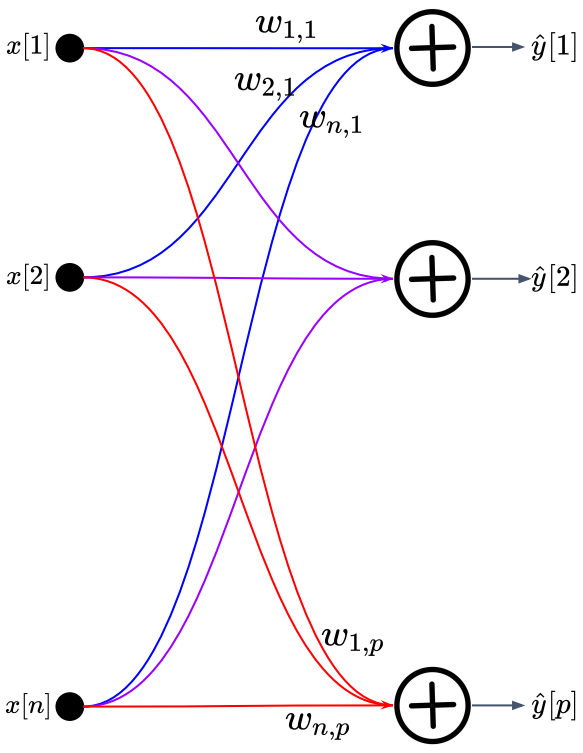



Figure 5: To predict $p$ outputs, the multivariate linear regression realizes a network with $n \times p$ parameters.

**Sigmoid function**

The very first thing that we are going to do in order to tackle the problem of binary classification is to make use of the **Sigmoid** function $\sigma : \mathbb{R} \to [0, 1]$ defined as

$$\sigma(x) = \frac{1}{1 + \exp(-x)}. \tag{3.1}$$

What this function does for us is to map real numbers to probability values (*i.e.*, positive numbers in the range $[0, 1]$). In Figure 6, we plot the Sigmoid function.
To understand the idea behind the use of the Sigmoid function, recall that the range of output in linear regression is not in our hands. With sigmoid, we alleviate that problem. Of course, this is not the only reason that we use the sigmoid function but let's keep things simple for now.

### 3.1. Cross-Entropy loss

Employing the sigmoid function along with the squared loss has been proven to be not suitable for binary classification problems. This is related to the optimization technique that we use, which has been shown to behave undesirably in this problem. To alleviate the

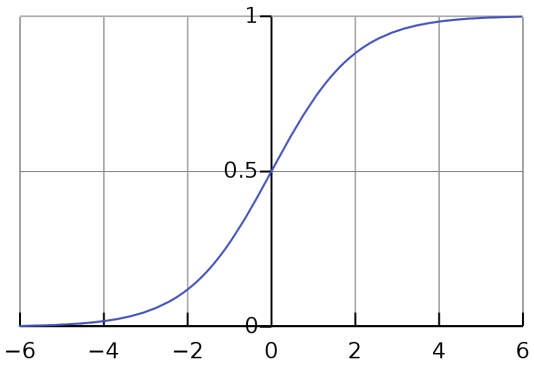



Figure 6: The Sigmoid function.

issue, we make use of the cross entropy (CE) loss defined as[8]

$$\ell_{\text{CE}}(\hat{y}, y) = -y \log\left(\hat{y}\right) - (1-y)\log\left(1-\hat{y}\right) . \tag{3.2}$$

As a reminder, the setting for the CE loss is binary classification with $y \in \{0, 1\}$ and $\hat{y} \in [0, 1]$. To understand Equation (3.2), assume $y = 1$. In this case, the loss will be[9]

$$\ell_{\text{CE}}(\hat{y}, y = 1) = -\log\left(\hat{y}\right) .$$

Since $\hat{y} \in [0, 1]$, $\log\left(\hat{y}\right) \leq 0$ and as such, the minimum of $\ell_{\text{CE}}(\hat{y}, y = 1) = -\log\left(\hat{y}\right) \geq 0$ will happen at $\hat{y} = 1$ because $\ell_{\text{CE}}(\hat{y} = 1, y = 1) = 0$. Conversely, assume $y = 0$. In this case,

$$\ell_{\text{CE}}(\hat{y}, y = 0) = -\log\left(1-\hat{y}\right) .$$

With a very similar line of reasoning, you can deduce that the minimizer will be $\hat{y} = 0$ (why?). This is exactly what we want from a loss function for a classification problem, as the minimum of the loss happens if and only if $\hat{y} = y$.

**Example 3** *Compute the gradient of the logistic w/ CE loss for a sample* $(\boldsymbol{x}, y)$ *defined as*

$$L = -y \log\left(\sigma(\boldsymbol{w}^\top \boldsymbol{x})\right) - (1-y)\log\left(1-\sigma(\boldsymbol{w}^\top \boldsymbol{x})\right)$$

*w.r.t.* $\boldsymbol{w}$.

**Solution.** We will use the chain-rule to simplify the derivation. First, note that $\frac{\partial}{\partial x}\log\left(f(x)\right) = \frac{f'(x)}{f(x)}$. Define $z = \sigma(\boldsymbol{w}^\top \boldsymbol{x})$. Note that,

$$\frac{\partial}{\partial \boldsymbol{w}}\sigma(\boldsymbol{w}^\top \boldsymbol{x}) = z(1-z)\boldsymbol{x} .$$

[8] We use log to denote the natural logarithm.

[9] A subtle point that can be easily taken care of in the code is that for CE loss, we assume $0 \times \log(0) = 0$.

we are interested in computing

$$\nabla_{\boldsymbol{w}} L = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_n} \end{pmatrix} = \begin{pmatrix} \frac{\partial L}{\partial z}\frac{\partial z}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial z}\frac{\partial z}{\partial w_n} \end{pmatrix} ,$$

We have,

$$\frac{\partial}{\partial \boldsymbol{w}} y \log\big(\sigma(\boldsymbol{w}^\top \boldsymbol{x})\big) = y \frac{\frac{\partial}{\partial \boldsymbol{w}}\sigma(\boldsymbol{w}^\top \boldsymbol{x})}{\sigma(\boldsymbol{w}^\top \boldsymbol{x})} = y\frac{z(1-z)\boldsymbol{x}}{z} = y(1-z)\boldsymbol{x} .$$

Similarly,

$$\frac{\partial}{\partial \boldsymbol{w}}(1-y)\log\big(1-\sigma(\boldsymbol{w}^\top \boldsymbol{x})\big) = (1-y)\frac{-\frac{\partial}{\partial \boldsymbol{w}}\sigma(\boldsymbol{w}^\top \boldsymbol{x})}{1-\sigma(\boldsymbol{w}^\top \boldsymbol{x})} = (1-y)\frac{-z(1-z)\boldsymbol{x}}{1-z} = -(1-y)z\boldsymbol{x} .$$

Therefore,

$$\nabla_{\boldsymbol{w}} L = -y(1-z)\boldsymbol{x} + (1-y)z\boldsymbol{x} = (z-y)\boldsymbol{x} .$$

This is like magic! We took a somewhat complicated formula for the logistic activation function, combined it with a somewhat complicated formula for the cross-entropy loss, and end up with a stunningly simple formula for the loss derivative! Observe that this is exactly the same formula as for $\partial \mathcal{L}_{\text{SE}}/\partial \boldsymbol{w}$ in the case of linear regression[10].

**Final Touch.** We can write the logistic function with CE loss for $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), \cdots, (\boldsymbol{x}_m, y_m)\}$ as

$$\mathcal{L}_{\text{CE}}(\boldsymbol{w}, \mathcal{D}) = -\frac{1}{m}\sum_{i=1}^{m}\Big\{ y_i \log(\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big)) + (1-y_i)\log(1-\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big))\Big\}. \tag{3.3}$$

This not very friendly looking error function deserves discussions. Recall that $y_i$ is either zero or one. Assume, $y_i = 1$. Then, for such a specific sample, the error function becomes

$$\begin{aligned} \mathcal{L}_{\text{CE}}(\boldsymbol{w}, (\boldsymbol{x}_i, y_i = 1)) &= -\Big\{ y_i \log(\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big)) + (1-y_i)\log(1-\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big))\Big\}_{y_i=1} \\ &= -\Big\{ 1 \times \log(\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big)) + 0 \times \log(1-\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big))\Big\} \\ &= -\log(\sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big)). \end{aligned} \tag{3.4}$$

Now recall that the image of the Sigmoid function is $[0, 1]$ and hence

$$\begin{aligned} & 0 \le \sigma(\boldsymbol{w}^\top \boldsymbol{x}_i) \le 1 \\ & \Rightarrow -\infty < \log\big(\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i)\big) \le 0 \\ & \Rightarrow 0 \le -\log\big(\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i)\big) < \infty. \end{aligned}$$

[10] Recall $\frac{\partial}{\partial \boldsymbol{w}}\frac{1}{2}\|\boldsymbol{w}^\top \boldsymbol{x} - y\|^2 = (\boldsymbol{w}^\top \boldsymbol{x} - y)\boldsymbol{x}$ .

This means that the minimum error of $-\log\big(\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i)\big)$ happens when $\log\big(\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i)\big) \to 0$ or $\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i) \to 1$. As such, for $y_i = 1$, our model has zero error if $\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i) \to 1$. In other words, for $y_i = 1$, we want our model to make a correct prediction of one (recall that $\sigma(\boldsymbol{w}^\top \boldsymbol{x}_i)$ is the prediction of our model for $\boldsymbol{x}_i$). A very similar argument can be made for $y_i = 0$. We leave this as an exercise for you.
If you want, now, you can design an artificial neuron with a sigmoid activation to tackle the problem of binary classification as shown in Figure 7. To learn the parameters of the model, the error in Equation (3.3) should be minimized. This can be done by following the steps depicted in Algorithm 4.

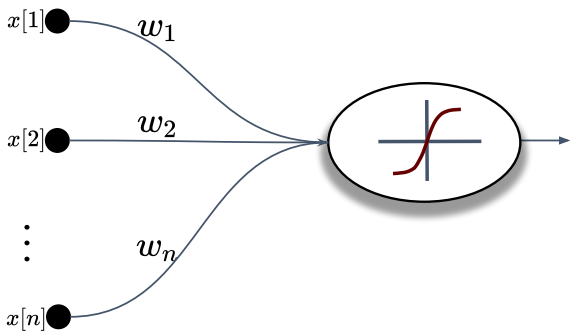



Figure 7: An artificial neuron with a sigmoid activation

**Algorithm 4:** Logistic Regression

**input :** $m$ data pairs, $\mathcal{D} = \big\{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_m, y_m)\big\}$ with $\boldsymbol{x}_i \in \mathbb{R}^n$ and $y_i \in \{0, 1\}$. The learning rate $\eta \in (0, 1]$.
**output:** $\boldsymbol{w}^*$, the parameters of the logistic model $\hat{y} = \sigma\big(\boldsymbol{w}^{*\top} \boldsymbol{x}\big)$ where $\sigma$ denotes the Sigmoid function (see Equation (3.1))
$\boldsymbol{w} \leftarrow$ random initialization;
**while** *not converged* **do**
  $z_i \leftarrow \sigma\big(\boldsymbol{w}^\top \boldsymbol{x}_i\big)$;
  $\nabla_{\boldsymbol{w}} \mathcal{L}_{\text{CE}} \leftarrow \frac{1}{m} \sum_{i=1}^{m} (z_i - y_i) \boldsymbol{x}_i$ ;
  $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} \mathcal{L}_{\text{CE}}$;
**end**
$\boldsymbol{w}^* \leftarrow \boldsymbol{w}$;

## 4. Softmax Classifier

So far we have focused on binary classification. For example, the logistic regression produces a scalar between 0 to 1.0, representing the probability of a binary decision (*e.g.*, 0.8 from a cat-dog image classifer suggests that the logistic classifier has a confidence of 80% that the image is a cat image and 20% chance of being a dog). However, most classification

problems involve more than two categories. Fortunately, this does not require any new ideas: everything pretty much works by analogy with the binary case.

**One-hot Coding** Consider the problem of classifying Iris flowers. In the iris dataset, each flower is encoded by 4 features, namely the length and width of its petal and sepal (see Figure 8). Furthermore, each image belongs to one of the following three categories: "Iris Versicolor", "Iris Setosa", "Iris Virginica". The first question we need to answer is: *"how can we represent the labels?"*. Perhaps the most natural impulse would be to choose $y \in \{1, 2, 3\}$, where the integers represent Iris Versicolor, Iris Setosa, Iris Virginica, respectively. If the categories had some natural ordering among them, say if we were trying to predict baby, toddler, adolescent, young adult, adult, geriatric, then it might even make sense to cast this problem as regression and keep the labels in this format.

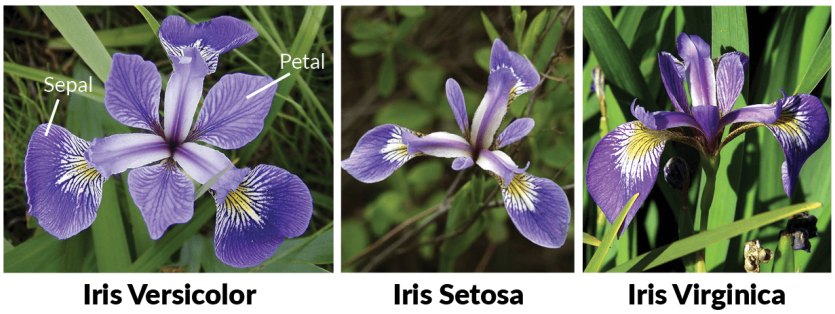



Figure 8: The Iris dataset contains 150 examples of three types of the Iris flower.

However and in general, classification problems do not come with natural orderings among the classes. A convenient work-around here is to represent labels by **one-hot vector**[11]. A one-hot encoding is a vector with as many components as the number of categories. The component corresponding to particular instance's category is set to 1 and all other components are set to 0. In our case, a label $\boldsymbol{y}$ would be a three-dimensional vector, with $(1, 0, 0)^\top$ corresponding to "Iris Versicolor", $(0, 1, 0)^\top$ to "Iris Setosa", and $(0, 0, 1)^\top$ to "Iris Virginica". That is a label is a member of the following set

$$\mathcal{Y} \triangleq \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right\} \tag{4.1}$$

Now we can design a linear model to address the multiclass problem by analogy with the binary case. We assume that the data is in the form

$$\mathcal{D} = \left\{ (\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \cdots, (\boldsymbol{x}_m, \boldsymbol{y}_m) \right\},$$

where $\boldsymbol{x}_i \in \mathbb{R}^n$ and $\boldsymbol{y}_i \in \{0, 1\}^K$ represents the one-hot coded label for $K$ classes. In essence, what we need to do is to design a function $f : \mathbb{R}^n \to \mathbb{R}^K$ and make sure that the predictions

[11] In the literature, sometimes it is also called a one-of-K encoding

of this function, *i.e.*, $f(\boldsymbol{x}_i)$ is as close as possible to $\boldsymbol{y}_i$. The simplest form that we can imagine here[12] is:

$$f(\boldsymbol{x}) = \boldsymbol{W}^\top \boldsymbol{x} \ . \tag{4.2}$$

Here, $\boldsymbol{W} \in \mathbb{R}^{n \times K}$ is a weight matrix. Now, note that $\boldsymbol{z} = f(\boldsymbol{x})$ can take any value in $\mathbb{R}^K$ (*e.g.*, it can take negative values). What, we are really interested in is to make sure that $\boldsymbol{z}$ becomes a probability vector. Let us recall the definition of a probability vector one more time[13].

**Definition 2 (Probability vector)** *We call a vector* $\boldsymbol{p} = (p_1, p_2, \cdots, p_K)^\top$ *a probability vector if*

$$\begin{aligned} & p_i \geq 0 \\ & \sum_{i=1}^{K} p_i = 1 \ . \end{aligned}$$

There is a multivariate generalization of the Sigmoid function called the **softmax** function. In particular, assume you have $\mathbb{R}^K \ni \boldsymbol{z} = \left(z_1, z_2, \cdots, z_K\right)^\top$. The softmax function accepts $\boldsymbol{z}$ and creates a probability vector $\boldsymbol{p} = \left(p_1, p_2, \cdots, p_K\right)^\top$ according to:

$$p_r = \frac{\exp(z_r)}{\sum_{i=1}^{K} \exp(z_i)} \tag{4.3}$$

$$\Rightarrow \boldsymbol{p} = \begin{pmatrix} p_1 \\ p_2 \\ \vdots \\ p_K \end{pmatrix} = \frac{1}{\sum_{i=1}^{K} \exp(z_i)} \begin{pmatrix} \exp(z_1) \\ \exp(z_2) \\ \vdots \\ \exp(z_K) \end{pmatrix} \ . \tag{4.4}$$

Let us dig more into the softmax function. First notice that the exponential function converts a real value to a positive number. That is why we use $\exp(\cdot)$ to convert elements of $\boldsymbol{z}$ to positive values. Regardless, we are not still there as we want a probability vector where the sum of all elements of the vector adding up to one. In doing so, we can simply normalize the result of the exponential function as done in the denominator in Equation (4.4).
With the softmax at our disposal, we can now define a loss to learn the parameters of the linear model in Equation (4.2). The loss that we widely use in conjunction with the softmax is called **Negative Log Likelihood** or NLL for short and is defined as

$$\ell_{\text{NLL}}\left(\hat{\boldsymbol{y}}, \boldsymbol{y}\right) \triangleq -\sum_{i=1}^{K} y[i] \log\left(\hat{y}[i]\right) = -\boldsymbol{y}^\top \log\left(\hat{\boldsymbol{y}}\right) \ . \tag{4.5}$$

[12] Aside from $f(\boldsymbol{x}) = \boldsymbol{b}$ where $\boldsymbol{b}$ is a constant vector (*e.g.*, $\boldsymbol{b} = \mathbb{E}(\boldsymbol{y})$).

[13] First norm of a vector, denoted by $\|\cdot\|_1$ is defined as $\|\boldsymbol{x}\|_1 = \sum_i |x_i|$ , with $x_i$ representing the element at location $i$ of the vector.

Here, $\boldsymbol{y} = \big(y[1], y[2], \cdots, y[K]\big)$ is the one-hot label and $\hat{\boldsymbol{y}} = \big(\hat{y}[1], \hat{y}[2], \cdots, \hat{y}[K]\big)$ is the prediction of the model and is a probability vector. That is[14]

$$\begin{aligned}\boldsymbol{z} &= \boldsymbol{W}^\top \boldsymbol{x} \\ \hat{\boldsymbol{y}} &= \text{softmax}(\boldsymbol{z})\end{aligned}$$

To understand the NLL better, recall that only one entry of the vector $\boldsymbol{y}$ is non-zero. Without loss of generality, assume the element r is non-zero, *i.e.*, $y[r] = 1$ and $y[i] = 0, i \neq r$. Let us show this label by $\boldsymbol{y}^\dagger = \big(0, \cdots, 0, y[r] = 1.0, \cdots, 0\big)^\top$. Plugging this into Equation (4.5) gives us

$$\ell_{\text{NLL}}\big(\hat{\boldsymbol{y}}, \boldsymbol{y}^\dagger\big) = -\log\big(\hat{y}[r]\big) \,.$$

Since, $0 \leq \hat{y}[r] \leq 1$ (recall that $\hat{\boldsymbol{y}}$ is the output of the softmax and a probability vector),

$$0 \leq \hat{y}[r] \leq 1 \Rightarrow \log(\hat{y}[r]) \leq 0 \Rightarrow -\log(\hat{y}[r]) \geq 0 \,.$$

As our goal is to minimize NLL, this means that we seek to achieve $-\log(\hat{y}[r]) \to 0$ or $\hat{y}[r] \to 1$. In words, the NLL would be extremely happy (zero loss) if $\hat{y}[r] = y[r] = 1$ for any construction of the one-hot vector $\boldsymbol{y}$. Since $\hat{\boldsymbol{y}}$ is a probability vector, if its $r$-th element tends to 1, then all the other element should go to zero and hence the ideal case would be if $\hat{\boldsymbol{y}}$ became a one-hot vector where its $r$-th element matches the input, hence $\hat{\boldsymbol{y}} = \boldsymbol{y}$ if we can achieve zero loss! The total loss of our model is hence:

$$\mathcal{L}_{\text{NLL}}\big(\boldsymbol{W}, \mathcal{D}\big) \triangleq -\frac{1}{m}\sum_{i=1}^{m} \boldsymbol{y}_i^\top \log\big(\hat{\boldsymbol{y}}_i\big) \,. \tag{4.6}$$

To update the weights of the model $\boldsymbol{W}$, define $L \triangleq -\boldsymbol{y}^\top \log\big(\hat{\boldsymbol{y}}\big)$ where $\hat{y} = \text{softmax}\big(\boldsymbol{W}^\top \boldsymbol{x}\big)$. Again with the help of the chain-rule, we can obtain

$$\nabla_{\boldsymbol{W}} L = \boldsymbol{x}\big(\hat{\boldsymbol{y}} - \boldsymbol{y}\big)^\top \,. \tag{4.7}$$

We leave this as an exercise for you. With the help of Equation (4.8), the gradient of the NLL can be easily obtained as

$$\nabla_{\boldsymbol{W}} \mathcal{L}_{\text{NLL}} = \frac{1}{m}\sum_{i=1}^{m} \boldsymbol{x}_i\big(\hat{\boldsymbol{y}}_i - \boldsymbol{y}_i\big)^\top \,. \tag{4.8}$$

Now that we have the gradient ready per Equation (4.8), we can make use of the iterative optimization techniques such as gradient descent to find the optimal values of the model parameters $\boldsymbol{W}$. This is illustrated using in Algorithm 5.

[14] As usual, we do not explicitly consider the bias. That is, instead of modeling in the form $\boldsymbol{z} = \boldsymbol{W}^\top \boldsymbol{x} + \boldsymbol{b}$, we model as $\boldsymbol{z} = \boldsymbol{W}^\top \boldsymbol{x}$. If a bias term is needed, we will augment our data accordingly.

**Algorithm 5: Training.** Softmax classifier

**input :** $m$ data pairs, $\mathcal{D} = \big\{(\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \cdots, (\boldsymbol{x}_m, \boldsymbol{y}_m)\big\}$ with $\boldsymbol{x}_i \in \mathbb{R}^n$ and $\boldsymbol{y}_i \in \{0,1\}^K$ being the one-hot vector representing the label of $\boldsymbol{x}_i$. The learning rate $\eta \in (0, 1]$.

**output:** $\boldsymbol{W}^*$, the parameters of the logistic model $\hat{\boldsymbol{y}} = \text{softmax}\big({\boldsymbol{W}^*}^\top \boldsymbol{x}\big)$

$\boldsymbol{W} \leftarrow$ random initialization;

**while** *not converged* **do**

$\hat{\boldsymbol{y}}_i \leftarrow \text{softmax}\big(\boldsymbol{W}^\top \boldsymbol{x}_i\big)$;

$\nabla_{\boldsymbol{W}} \mathcal{L}_{\text{NLL}} \leftarrow \frac{1}{m} \sum_{i=1}^{m} (\hat{\boldsymbol{y}}_i - \boldsymbol{y}_i)\boldsymbol{x}_i$ ;

$\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \nabla_{\boldsymbol{W}} \mathcal{L}_{\text{NLL}}$;

**end**

$\boldsymbol{W}^* \leftarrow \boldsymbol{W}$;

**Understanding the training.**

In Figure 9, an illustration of the softmax classifier is provided. Therein and in the top panel, the functionality of the softmax, from its input $\boldsymbol{x} = \big(x[1], x[2], \cdots, x[n]\big)^\top$ to its predictions $\hat{\boldsymbol{y}} = \big(\hat{y}[1], \hat{y}[2], \cdots, \hat{y}[K]\big)^\top$ is shown. In the bottom of Figure 9, we dissect the softmax classifier for an image classification problem. Therein, we assume that we are solving a problem with three classes (represented by blue, green and red colors). Given an input image, the dot product between each row of the matrix $\boldsymbol{W}^\top$ and the input image identifies the score of the corresponding class. The score (logit) is a real value. Once the scores are gone through the softmax function, we will get a probability vector and will use it to either make predictions (test stage) or update the parameters of the model (training stage).

© M. Harandi.

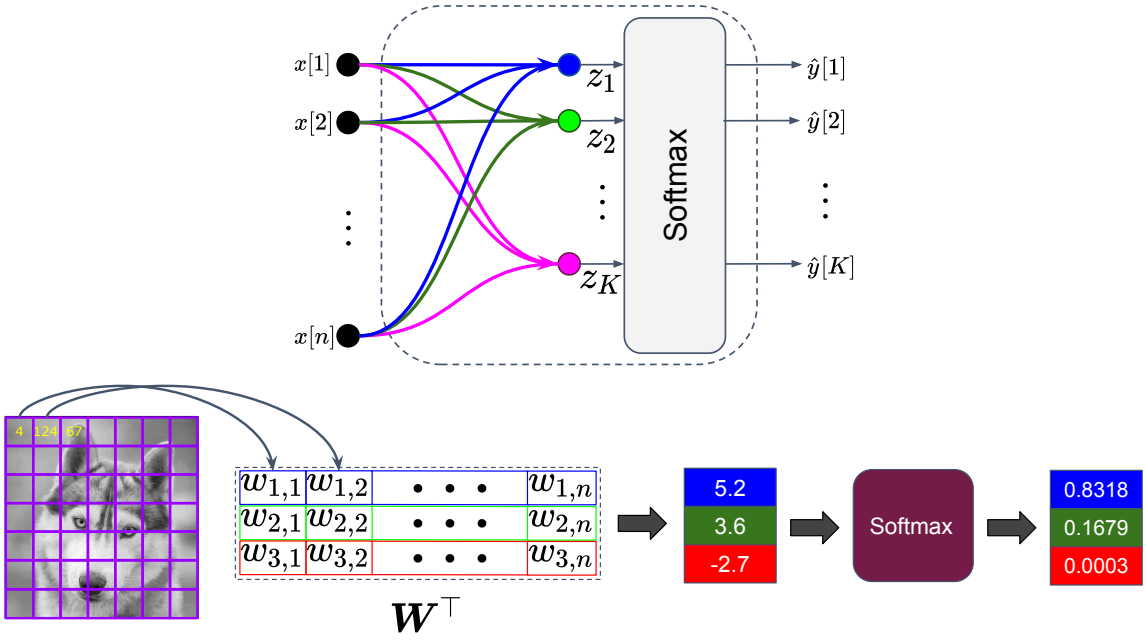



Figure 9: Illustration of a softmax classifier. **top** A softmax classifier accepts an $n$-dimensional input $\boldsymbol{x}$ and linearly transform it to a $K$-dimensional logit scores. The softmax function converts the logits into a notion of probabilities. **bottom** Dissecting the softmax classifier using image classification as an example (see the text for details).

## Appendix A. Derivations

### A.1. Linear Regression

Let $\{(x_1, y_1), (x_2, y_2), \cdots, (x_m, y_m)\}$ be $m$ points with $x_i, y_i \in \mathbb{R}$. We want to learn a linear model with parameters $w$ and $b \in \mathbb{R}$ in the form

$$\hat{y} = wx + b \, . \tag{A.1}$$

We can define the error of such a model as

$$\begin{aligned} \mathcal{L}_{\text{SE}}(w, b) &\triangleq \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2 \\ &= \frac{1}{m} \sum_{i=1}^{m} (wx_i + b - y_i)^2 \, . \end{aligned} \tag{A.2}$$

To obtain the optimal values of the parameters $w$ and $b$, we need to solve

$$\frac{\partial \mathcal{L}_{\text{SE}}(w, b)}{\partial w} = 0 \tag{A.3}$$

$$\frac{\partial \mathcal{L}_{\text{SE}}(w, b)}{\partial b} = 0 \tag{A.4}$$

We start by Equation (A.3);

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SE}}(w, b)}{\partial w} &= \frac{\partial}{\partial w} \left\{ \frac{1}{m} \sum_{i=1}^{m} (wx_i + b - y_i)^2 \right\} \\ &= \frac{1}{m} \sum_{i=1}^{m} \frac{\partial}{\partial w} \left\{ (wx_i + b - y_i)^2 \right\} \\ &= \frac{1}{m} \sum_{i=1}^{m} 2x_i (wx_i + b - y_i) \, . \end{aligned}$$

As such;

$$\begin{aligned} &\frac{1}{m} \sum_{i=1}^{m} 2x_i (wx_i + b - y_i) = 0 \Rightarrow \sum_{i=1}^{m} wx_i^2 + \sum_{i=1}^{m} bx_i - \sum_{i=1}^{m} y_i x_i = 0 \\ &\Rightarrow w \sum_{i=1}^{m} x_i^2 = \sum_{i=1}^{m} y_i x_i - b \sum_{i=1}^{m} x_i = 0 \Rightarrow \boxed{w = \frac{\sum_{i=1}^{m} y_i x_i - b \sum_{i=1}^{m} x_i}{\sum_{i=1}^{m} x_i^2}} \, . \end{aligned}$$

Similarly,

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SE}}(w, b)}{\partial b} &= \frac{\partial}{\partial b} \left\{ \frac{1}{m} \sum_{i=1}^{m} (wx_i + b - y_i)^2 \right\} = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial}{\partial b} \left\{ (wx_i + b - y_i)^2 \right\} \\ &= \frac{1}{m} \sum_{i=1}^{m} 2(wx_i + b - y_i) \, , \end{aligned}$$

which leads to

$$\frac{1}{m}\sum_{i=1}^{m} 2(wx_i + b - y_i) = 0 \Rightarrow \sum_{i=1}^{m} wx_i + \sum_{i=1}^{m} b - \sum_{i=1}^{m} y_i = 0$$
$$\Rightarrow \boxed{b = \frac{\sum_{i=1}^{m} y_i - w\sum_{i=1}^{m} x_i}{m}}.$$

Now let us define the following,

$$\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i, \tag{A.5}$$

$$\bar{y} = \frac{1}{m}\sum_{i=1}^{m} y_i. \tag{A.6}$$

Also, let $\boldsymbol{x} = \big(x_1, x_2, \cdots, x_m\big)^\top$ and $\boldsymbol{y} = \big(y_1, y_2, \cdots, y_m\big)^\top$. Then we have,

$$\sum_{i=1}^{m} y_i x_i = \boldsymbol{x}^\top \boldsymbol{y},$$
$$\sum_{i=1}^{m} x_i^2 = \boldsymbol{x}^\top \boldsymbol{x}.$$

With the above,

$$\boxed{b = \bar{y} - w\bar{x}} \tag{A.7}$$

$$\boxed{w = \frac{\boldsymbol{x}^\top \boldsymbol{y} - m \times b \times \bar{x}}{\boldsymbol{x}^\top \boldsymbol{x}}} \tag{A.8}$$

Now plugging $b$ into $w$ leads to

$$w = \frac{\boldsymbol{x}^\top \boldsymbol{y} - m \times (\bar{y} - w\bar{x}) \times \bar{x}}{\boldsymbol{x}^\top \boldsymbol{x}} = \frac{\boldsymbol{x}^\top \boldsymbol{y} - m\bar{x}\bar{y} + mw\bar{x}^2}{\boldsymbol{x}^\top \boldsymbol{x}}$$
$$\Rightarrow w^* = \frac{\boldsymbol{x}^\top \boldsymbol{y} - m\bar{x}\bar{y}}{\boldsymbol{x}^\top \boldsymbol{x} - m\bar{x}^2}$$

Once we found $w$, we can plug it back into Equation (A.7) to obtain $b$.

### A.2. Multivariate Linear Regression

We will consider the most general case and derive the solution for a linear mapping in the form $f : \mathbb{R}^n \to \mathbb{R}^p,\ f(\boldsymbol{x}) = \boldsymbol{W}^\top \boldsymbol{x}$. We assume, $m$ pairs of data are available to us and show it by $\mathcal{D} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \cdots, (\boldsymbol{x}_m, \boldsymbol{y}_m)\}$ where $\boldsymbol{x}_i \in \mathbb{R}^n$ and $\boldsymbol{y}_i \in \mathbb{R}^p$. The error of the model is

$$\mathcal{L}_{\text{SE}}(\boldsymbol{W}) = \sum_{i=1}^{m} \left\|\boldsymbol{W}^\top \boldsymbol{x}_i - \boldsymbol{y}_i\right\|^2, \tag{A.9}$$

where $\|\cdot\|^2$ denotes the norm-2 of a vector[15] and $\boldsymbol{W} \in \mathbb{R}^{n\times p}$ is the parameter of the model. Define,

$$\boldsymbol{X} = \begin{pmatrix} x_1[1], & x_1[2], & \cdots & x_1[n] \\ x_2[1], & x_2[2], & \cdots & x_2[n] \\ \vdots & \vdots & & \vdots \\ x_m[1], & x_m[2], & \cdots, & x_m[n] \end{pmatrix}, \tag{A.10}$$

and

$$\boldsymbol{Y} = \begin{pmatrix} y_1[1], & y_1[2], & \cdots & y_1[p] \\ y_2[1], & y_2[2], & \cdots & y_2[p] \\ \vdots & \vdots & & \vdots \\ y_m[1], & y_m[2], & \cdots & y_m[p] \end{pmatrix}. \tag{A.11}$$

As such $\boldsymbol{X} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{Y} \in \mathbb{R}^{m\times p}$. Now, we can rewrite Equation (A.9) as

$$\mathcal{L}_{\text{SE}}(\boldsymbol{W}) = \sum_{i=1}^{m} \left\|\boldsymbol{W}^\top \boldsymbol{x}_i - \boldsymbol{y}_i\right\|^2 = \left\|\boldsymbol{X}_{m\times n}\boldsymbol{W}_{n\times p} - \boldsymbol{Y}_{m\times p}\right\|_F^2. \tag{A.12}$$

In Equation (A.12) the dimensionality of matrices is shown as subscripts to assist the reader. The notation $\|\cdot\|_F$ denotes the Frobenius norm of a matrix and is defined as;

$$\left\|\boldsymbol{A}_{r\times s}\right\|_F^2 = \sum_{i=1}^{r}\sum_{j=1}^{s} a[i,j]^2 = \text{Tr}\left(\boldsymbol{A}^\top \boldsymbol{A}\right).$$

One can show that (see for example section 2.5 of (**?**)) $\nabla_{\boldsymbol{W}} \mathcal{L}_{\text{SE}}(\boldsymbol{W}) = 2\boldsymbol{X}^\top\left(\boldsymbol{X}\boldsymbol{W} - \boldsymbol{Y}\right)$, and hence

$$\begin{aligned} \nabla_{\boldsymbol{W}} \mathcal{L}_{\text{SE}}(\boldsymbol{W}) = 0 &\Rightarrow 2\boldsymbol{X}^\top\left(\boldsymbol{X}\boldsymbol{W} - \boldsymbol{Y}\right) = 0 \\ &\Rightarrow \boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{W} - \boldsymbol{X}^\top\boldsymbol{Y} = 0 \\ &\Rightarrow \boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{W} = \boldsymbol{X}^\top\boldsymbol{Y} \\ &\Rightarrow \boxed{\boldsymbol{W} = \left(\boldsymbol{X}^\top\boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top\boldsymbol{Y}.} \end{aligned}$$

[15] Recall that for a vector $\boldsymbol{x} \in \mathbb{R}^n$, the norm-2 is defined as $\|\boldsymbol{x}\|^2 = \boldsymbol{x}^\top\boldsymbol{x} = \sum_{j=1}^{n} x[j]^2$ with $x[j]$ denoting the j-th element of $\boldsymbol{x}$.